



1 Objetivo

O objetivo deste trabalho é implementar um simulador de escalonamento de CPU para avaliar o desempenho de diferentes políticas de gerenciamento de processos. O simulador deverá calcular métricas como Tempo de Espera Médio, Tempo de Retorno (*Turnaround*) Médio e Vazão (*Throughput*).

2 Linguagens Permitidas

O trabalho deve ser desenvolvido obrigatoriamente em uma das seguintes linguagens: **Java, C ou C++**.

3 Algoritmos a Implementar

Os grupos deverão implementar as seguintes lógicas de escalonamento:

1. **FCFS (First-Come, First-Served):** Escalonamento não-preemptivo simples por ordem de chegada.
2. **SRTF (Shortest Remaining Time First):** Variante preemptiva do SJF. O escalonador sempre escolhe o processo que possui o menor tempo de execução restante.
3. **Round-Robin com Quantum por Predição:** Escalonamento circular onde o valor do *quantum* é recalculado a cada troca de contexto para ser igual à **menor média exponencial** (τ) entre os processos na fila de prontos. Considere $\alpha = 0.5$, $\tau_0 = 10ms$.
4. **Multilevel Queue (MLQ):** Implementar duas filas estáticas com prioridades fixas. A Fila 1 (Alta Prioridade) deve ser escalonada via Round-Robin (quantum fixo), e a Fila 2 (Baixa Prioridade) via FCFS (*First-Come, First-Served*). Processos na Fila 2 só executam se a Fila 1 estiver vazia.

4 Fundamentação: Média Exponencial

Para o algoritmo de Round-Robin sugerido, o sistema deve "prever" a duração do próximo surto de CPU de cada processo para ajustar o ritmo do escalonador. Essa predição será calculada pela fórmula da **Média Exponencial**:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

Definições:

- t_n : Duração real do último surto de CPU completado pelo processo.
- τ_{n+1} : Valor previsto para o próximo surto de CPU.
- α : Constante de peso (deve-se utilizar $\alpha = 0.5$).
- τ_0 : Valor inicial de predição (utilizar $10ms$).

Impacto da Adoção: A utilização desta fórmula permite que o escalonador identifique processos interativos (que fazem muito I/O e possuem surtos curtos). Ao definir o *quantum* como o menor τ da fila, o sistema garante que processos rápidos terminem sua tarefa e liberem a CPU sem sofrerem interrupções desnecessárias, aumentando a responsividade.

5 Formato do Arquivo de Entrada

O simulador deverá ler um arquivo `processos.txt`. Como processos interativos realizam múltiplas chamadas de sistema, o formato deve permitir N instantes de I/O.

PID;Chegada;Burst_Total;Prioridade;[Instantes_de_I/O]

O campo `Instantes_de_I/O` é uma lista separada por vírgulas que indica em quais momentos do tempo de execução (tempo de CPU acumulado) o processo solicita E/S. Considere que cada operação de I/O bloqueia o processo por um tempo fixo de **5 unidades de tempo**.

Exemplo: 101;0;40;2;10,25

- O processo 101 chega no tempo 0 e precisa de 40ms totais de CPU.
- Ele fará o 1º I/O após usar 10ms de CPU. Ao retornar do I/O, ele precisará de mais 30ms.
- Ele fará o 2º I/O após usar um total acumulado de 25ms de CPU.

6 Regras de Avaliação e Apresentação

- **Formação dos Grupos:** Os grupos serão constituídos por até **4 alunos**.
- **Apresentação:** Haverá um dia dedicado exclusivamente à apresentação dos trabalhos.
- **Arguição:** No dia da apresentação, o professor realizará um sorteio de **dois alunos** do grupo.
- **Responsabilidade:** Os alunos sorteados deverão apresentar as implementações, explicar a arquitetura do código e responder às dúvidas do professor sobre as escolhas técnicas adotadas.
- **Nota Individual:** Caso um aluno sorteado não esteja presente no momento da arguição, ele receberá automaticamente **nota 0 (zero)** no trabalho, sem possibilidade de reposição, salvo justificativa médica oficial.
- **Dependência de Nota do Grupo:** A nota atribuída ao grupo será **diretamente dependente da qualidade da explicação e defesa** técnica realizada pelos alunos sorteados. Espera-se que todos os membros dominem integralmente a implementação e a teoria envolvida.

7 Entrega

A entrega deve conter o código-fonte comentado, um arquivo **README** com instruções de compilação e um breve relatório comparando os resultados obtidos pelos quatro algoritmos para o mesmo arquivo de entrada.